

CSE 715: Computer Graphics

Chapter 3 — Scan Conversion

Complete Model Answers (2020–2024)

University of Chittagong — 7th Semester B.Sc. (Engg.) Examination

Source: Schaum's Outline of Computer Graphics (Xiang & Plastock), Chapter 3, pp.25–67

Contents

1	Theoretical Framework — Reusable Across All Years	2
1.1	Scan-Converting a Line	2
1.2	Scan-Converting a Circle	2
1.3	Region Filling	3
1.4	Anti-Aliasing	3
1.5	Recursively-Defined (Fractal) Curves	4
2	Examination 2024 — Q3(c) (Section A)	4
3	Examination 2023 — Q1(c), Q2(a–d) (Section A)	6
4	Examination 2022 — Q2(a–d) (Section A)	7
5	Examination 2021 — Q1(f), Q2(a–b) (Section A)	9
6	Examination 2020 — Q2(d), Q3(b–c), Q4(c) (Group-A)	11
7	Quick Summary — Exam Patterns for Chapter 3	13

Theoretical Framework — Reusable Across All Years

This chapter cluster shows up as **Q2 (2022/2023) or Q3 (2024) in Section A** every year — the exact question number shifts, but the content is always the same four ideas: line scan-conversion (DDA/Bresenham), circle scan-conversion (midpoint/Bresenham + 8-way symmetry), anti-aliasing artifacts, region filling, and recursively-defined (fractal) curves.

Scan-Converting a Line

A line is defined by two endpoints and $y = mx + b$. For $|m| \leq 1$ we step x by 1 and solve for y ; for $|m| > 1$ we step y by 1 and solve for x .

- ▷ **Direct line-equation method:** plug each integer x (or y) into $y = mx + b$ — needs floating-point multiply every step.
- ▷ **DDA (Digital Differential Analyzer):** incremental — $y_{i+1} = y_i + m$ (if $|m| \leq 1$) or $x_{i+1} = x_i + 1/m$ (if $|m| > 1$). No multiply, but still a floating-point *add* each step, and cumulative rounding error can drift on long lines.
- ▷ **Bresenham’s algorithm:** integer-only decision variable d replaces the floating add. For $0 \leq m \leq 1$:

$$\Delta x = x_2 - x_1, \quad \Delta y = y_2 - y_1, \quad \text{Inc}_1 = 2\Delta y, \quad \text{Inc}_2 = 2(\Delta y - \Delta x), \quad d_1 = \text{Inc}_1 - \Delta x$$

Each step: if $d > 0$ choose the **upper** pixel ($y++$), $d += \text{Inc}_2$; else choose the **lower** pixel (same y), $d += \text{Inc}_1$. Always $x++$.

Why Bresenham beats DDA: it produces mathematically identical (correctly-rounded) pixel choices using only integer addition/subtraction (and $\times 2$, a bit-shift) — no floating-point multiply, no accumulated rounding drift on long lines.

Scan-Converting a Circle

- ▷ **8-way symmetry:** compute one point (x, y) in the 90–45 octant; the other 7 come free by reflection:

$$\begin{array}{ll} P_1 = (x, y) & P_5 = (-x, -y) \\ P_2 = (y, x) & P_6 = (-y, -x) \\ P_3 = (-y, x) & P_7 = (y, -x) \\ P_4 = (-x, y) & P_8 = (x, -y) \end{array}$$

Midpoint circle algorithm (integer, decision parameter p):

$$x = 0, \quad y = r, \quad p_1 = 1 - r$$

Each step: if $p < 0$, choose $T = (x+1, y)$, $p += 2x + 3$; else choose $S = (x+1, y-1)$, $p += 2(x - y) + 5$, $y--$. Always $x++$. Loop while $x \leq y$.

Bresenham's circle algorithm is algebraically equivalent (decision variable $d_i = 3 - 2r$ initially, updated by $+4x + 6$ or $+4(x-y) + 10$) — same pixel choices, different derivation (distance-squared vs. midpoint test). To center at (x_c, y_c) : compute the octant for a circle at the origin, then add (x_c, y_c) to every plotted point.

Region Filling

- ▷ **Boundary-fill:** recursive, starts at a seed, fills unless pixel = boundary color or already filled.
- ▷ **Flood-fill:** recursive, starts at a seed, fills unless pixel $\neq$ the region's original color.
- ▷ **4-connected vs. 8-connected:** 4-connected pixels share an edge (N/S/E/W); 8-connected pixels also share a corner (+ diagonals).
- ▷ **The consistency trap (critical for "will this fill work?" questions):** using the *same* connectivity rule for both the boundary and the interior is self-contradictory for a diagonal (staircase) boundary. A boundary drawn as a diagonal, single-pixel-wide staircase is only **4-connected** along its own length (consecutive boundary pixels touch corner-to-corner, not edge-to-edge, at the "steps"). If the fill algorithm is run with **8-connected** logic, it treats diagonal movement as valid and can **slip through the corner-gap** between two staircase-adjacent boundary pixels, leaking color into the exterior. The theoretically consistent pairing is: **8-connected boundary + 4-connected fill**, or **4-connected boundary + 8-connected fill** — never the same definition for both.
- ▷ **Scan-line polygon fill:** for a *geometrically*-defined polygon (vertex list), find all edge/scan-line intersections per row, sort by x , pair them up, and draw filled spans between pairs. Horizontal edges are skipped (auto-filled). An **edge list** (per edge: y_{min} , y_{max} , x at y_{min} , $1/m$) tracks active edges; y_{max} of the lower of two edges meeting at a local max/min vertex is decremented by 1 to avoid double-counting that vertex.

Anti-Aliasing

Three adverse effects of scan conversion (memorise this list):

1. **Staircase / jaggies** — stair-step appearance of lines/circles/filled-region borders.
2. **Unequal brightness** — a slanted line looks dimmer than a horizontal/vertical one at the same intensity, because its pixels are spaced ≈ 1.414 (i.e. $\sqrt{2}$) units apart along the line vs. 1 unit apart for horizontal/vertical — same "ink," lower density $\rightarrow$ perceived as dimmer.
3. **Picket fence problem** — an object whose true spacing isn't a multiple of the pixel pitch either drifts in overall length (*global aliasing*, snap-to-pixel) or in

even spacing (*local aliasing*, forced equal spacing) when scan-converted.

Fixing unequal brightness (how to "solve" the dimmer-slanted-line problem): boost the perceived intensity of slanted-line pixels to compensate for their lower spatial density — concretely via **anti-aliasing** techniques:

- ▷ **Area sampling** (pre-filter): intensity of a pixel $\propto$ % overlap between the pixel's cell and the (blurred) primitive.
- ▷ **Super sampling** (post-filter): subdivide each pixel into $n \times n$ subpixels; intensity $\propto$ fraction of subpixels inside the object.
- ▷ **Lowpass filtering** (post-filter): replace each pixel with a weighted average of itself and its neighbors using an $(2n+1) \times (2n+1)$ kernel whose weights sum to 1 (equal weights = box filter; Gaussian-shaped weights = Gaussian filter).
- ▷ **Pixel phasing** (hardware): physically nudge a pixel by a fraction of the inter-pixel distance toward the true line.

Recursively-Defined (Fractal) Curves

Two variants appear in past papers, both built by replacing a line's middle third with a bump and recursing on every resulting sub-segment:

	Classic Koch curve (triangular bump)	Quadratic Koch curve (square bump)
Replacement rule	middle third $\rightarrow$ 2 sides of an outward equilateral triangle	middle third $\rightarrow$ 3 sides of an outward square notch
Segments per edge per iteration	$\times 4$	$\times 5$
Turn angles	$+60, -120, +60$	$+90, -90, -90, +90$
Seen as	Koch snowflake (start from a triangle)	"square-wave" fractal (start from a line)

Segment-count rule (fast way to answer "how many segments in generation n ?): if the base shape has E edges, generation n has $E \times (\text{multiplier})^n$ segments — e.g. a triangle ($E = 3$) under the classic rule has 3×4^n segments at generation n (3, 12, 48, 192, ...).

Examination 2024 — Q3(c) (Section A)

Question: A line is to be drawn from $(2, 3)$ to $(10, 7)$. (1) Predict the next pixel using DDA if current pixel is $(2, 3)$, showing $\Delta x, \Delta y$. (2) Predict the next pixel using Bresenham if current pixel is $(2, 3)$, computing the initial decision parameter. (3) Continue step by step to $(10, 7)$, recording pixel coordinates for both algorithms. (4)

At each step, do DDA and Bresenham choose the same pixel? If not, explain why. [4 marks]

Setup

$\Delta x = 10 - 2 = 8$, $\Delta y = 7 - 3 = 4 \Rightarrow m = \Delta y / \Delta x = 0.5$ (so $|m| \leq 1$, step x by 1 each time).

(1)–(3) DDA trace

$y_{i+1} = y_i + m = y_i + 0.5$, pixel = Floor($y + 0.5$) each step:

$(2, 3) \rightarrow (3, 4) \rightarrow (4, 4) \rightarrow (5, 5) \rightarrow (6, 5) \rightarrow (7, 6) \rightarrow (8, 6) \rightarrow (9, 7) \rightarrow (10, 7)$

(2)–(3) Bresenham trace

$\text{Inc}_1 = 2\Delta y = 8$, $\text{Inc}_2 = 2(\Delta y - \Delta x) = 2(4 - 8) = -8$, $d_1 = \text{Inc}_1 - \Delta x = 8 - 8 = 0$.

d (entering step)	x	y	rule applied
0	2	3	start
0	3	3	$d \leq 0 \Rightarrow d += \text{Inc}_1 = 8$, same y
8	4	4	$d > 0 \Rightarrow d += \text{Inc}_2 = 0$, $y++$
0	5	4	$d \leq 0 \Rightarrow d += 8$, same y
8	6	5	$d > 0 \Rightarrow d += 0$, $y++$
0	7	5	same y
8	8	6	$y++$
0	9	6	same y
8	10	7	$y++$

$(2, 3) \rightarrow (3, 3) \rightarrow (4, 4) \rightarrow (5, 4) \rightarrow (6, 5) \rightarrow (7, 5) \rightarrow (8, 6) \rightarrow (9, 6) \rightarrow (10, 7)$

(4) Do they choose the same pixel? [key insight]

No. They agree only at $x = 2, 4, 6, 8, 10$ and **diverge at every odd step** ($x = 3, 5, 7, 9$: DDA picks the pixel one row higher than Bresenham). **Reason:** $m = 0.5$ puts the true line exactly *on the boundary* between two pixel rows at every other x (the y -values are exact half-integers: 3.5, 4.5, 5.5, 6.5) — a genuine geometric tie. DDA's rounding rule Floor($y + 0.5$) always rounds an exact .5 *up*. Bresenham's decision variable hits exactly $d = 0$ at those same steps, and its tie-breaking rule (this text's convention: $d \leq 0 \Rightarrow$ keep the *same* y , i.e. round down) resolves the tie the *opposite* way. Neither algorithm is "wrong" — $m = 0.5$ is simply the one slope value where

every step is an exact tie, and the two algorithms' tie-breaking conventions disagree.

Examination 2023 — Q1(c), Q2(a–d) (Section A)

Question breakdown:

- Q1(c) How does Bresenham's line algorithm overcome the limitations of the DDA algorithm? (1)
- Q2(a) Why do slanted lines look comparatively dimmer than straight lines? (1)
- Q2(b) Indicate raster locations chosen in the 3rd quadrant when scan-converting a circle $r = 10$ (center at origin) using the Midpoint circle algorithm. (4)
- Q2(c) Could you fill the given hexagon using flood-fill with 8-connected pixels? Explain with a diagram. (3)
- Q2(d) A type of Koch curve is drawn as shown (1st/2nd generation, square bump). Draw the 3rd generation. (1)

Q1(c) — Bresenham vs. DDA [1 mark]

See §1.1. Bresenham replaces DDA's per-step floating-point addition (and the resulting cumulative rounding drift on long lines) with an all-integer decision variable updated by addition/subtraction and a $\times 2$ shift — faster and drift-free.

Q2(a) — Why Slanted Lines Look Dimmer [1 mark]

See §1.4. Pixels on a 45 line are $\sqrt{2} \approx 1.414$ units apart vs. 1 unit apart on a horizontal/vertical line — same intensity spread over a sparser set of pixels reads as dimmer to the eye.

Q2(b) — Midpoint Circle, $r = 10$, 3rd Quadrant [4 marks]

First-octant trace ($x = 0, y = 10, p_1 = 1 - r = -9$; while $x \leq y$: if $p < 0$, $p += 2x + 3$; else $p += 2(x - y) + 5$, $y--$; always $x++$):

p (enter)	x	y	plotted	rule
−9	0	10	(0, 10)	$p < 0 \Rightarrow p += 3 \Rightarrow -6$
−6	1	10	(1, 10)	$p < 0 \Rightarrow p += 5 \Rightarrow -1$
−1	2	10	(2, 10)	$p < 0 \Rightarrow p += 7 \Rightarrow 6$
6	3	10	(3, 10)	$p \geq 0 \Rightarrow p += 2(3 - 10) + 5 = -9 \Rightarrow -3, y-- \rightarrow 9$
−3	4	9	(4, 9)	$p < 0 \Rightarrow p += 11 \Rightarrow 8$
8	5	9	(5, 9)	$p \geq 0 \Rightarrow p += 2(5 - 9) + 5 = -3 \Rightarrow 5, y-- \rightarrow 8$
5	6	8	(6, 8)	$p \geq 0 \Rightarrow p += 2(6 - 8) + 5 = 1 \Rightarrow 6, y-- \rightarrow 7$
6	7	7	(7, 7)	$x = y$: stop

Octant points: $(0, 10), (1, 10), (2, 10), (3, 10), (4, 9), (5, 9), (6, 8), (7, 7)$.

3rd-quadrant points ($x < 0, y < 0$) come from reflections $P_5 = (-x, -y)$ and $P_6 = (-y, -x)$:

$(-1, -10), (-2, -10), (-3, -10), (-4, -9), (-5, -9), (-6, -8), (-7, -7),$

$(-8, -6), (-9, -5), (-9, -4), (-10, -3), (-10, -2), (-10, -1)$

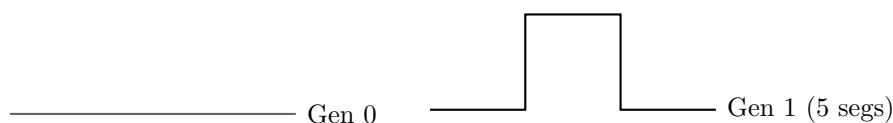
(13 unique points; $(0, -10)$ and $(-10, 0)$ lie exactly on the axis boundary and are shared with quadrants 2/4.)

Q2(c) — Flood-Fill a Hexagon, 8-Connected [3 marks]

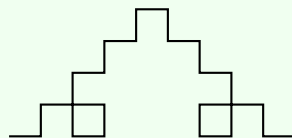
Not reliably. A hexagon has slanted (diagonal) edges. When a diagonal edge is scan-converted, its boundary pixels form a single-pixel-wide *staircase* — consecutive boundary pixels along that edge touch only **corner-to-corner** (8-connected), not edge-to-edge. If the fill itself is run with **8-connected** logic, it is allowed to step diagonally too, and can slip through the very same corner-gap the staircase boundary has, leaking out of the hexagon into the background (see the theoretical note in §1.3 — using the same connectivity definition for both boundary and interior is self-contradictory). **Fix:** pair an 8-connected boundary with a 4-connected fill (or vice versa) so the fill cannot pass through the diagonal gaps.

Q2(d) — Koch Curve, 3rd Generation (Square Bump) [1 mark]

Generation 1 (given, 5 segments of length $L/3$ each): right, up, right, down, right.



3rd generation = apply the same 5-segment rule to *each* of the 5 segments of generation 1 (i.e. 2 recursive applications from the base line) $\Rightarrow 5 \times 5 = 25$ segments:



Examination 2022 — Q2(a–d) (Section A)

Question breakdown:

- (a) What do you mean by the 8-way symmetry of a circle? Explain with a diagram. (1.5)
- (b) Indicate raster locations chosen by the Midpoint circle algorithm for radius 10, center (50, 50). (4)
- (c) Why do slanted lines look comparatively dimmer than straight lines? How do you solve this? (2)
- (d) A type of Koch curve is drawn from a triangle to a 6-pointed star (1st/2nd generation). Draw the 3rd generation. (1.5)

Q2(a) — 8-Way Symmetry [1.5 marks]

See §1.2. One computed point (x, y) in the 90–45 octant yields 7 more by reflecting about the axes and the 45 diagonal: $(x, y), (y, x), (-y, x), (-x, y), (-x, -y), (-y, -x), (y, -x), (x, -y)$.

Q2(b) — Midpoint Circle, $r = 10$, Center (50, 50) [4 marks]

Octant trace relative to the origin is identical to §3's table above (same $r = 10$): (0, 10), (1, 10), (2, 10), (3, 10), (4, 9), (5, 9), (6, 8), (7, 7).

Apply 8-way symmetry, then **translate every point by** (+50, +50) (i.e. replace `setPixel( $x, y$ )` with `setPixel( $x+50, y+50$ )`). E.g. octant points become

(50, 60), (51, 60), (52, 60), (53, 60), (54, 59), (55, 59), (56, 58), (57, 57)

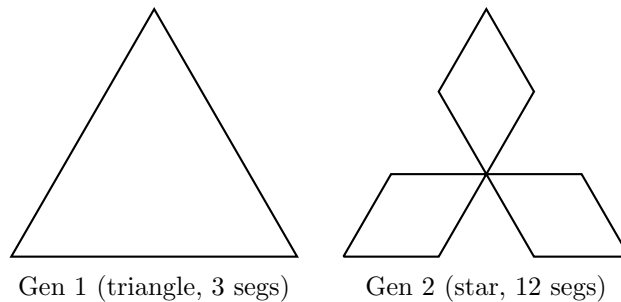
and the remaining 48 points of the full 56-point circle are the 7 symmetric reflections of each, each likewise shifted by (+50, +50).

Q2(c) — Why Dimmer + How to Fix [2 marks]

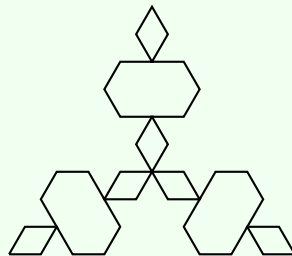
Why: see §1.4 — diagonal pixel spacing ($\sqrt{2}$ units) vs. axis-aligned spacing (1 unit) at equal intensity $\Rightarrow$ lower perceived density $\Rightarrow$ dimmer. **Fix:** apply an **anti-aliasing** technique — e.g. *area sampling* (weight each pixel by % overlap with the true line) or *super-sampling* (subdivide into subpixels, intensity $\propto$ fraction covered) — to boost the slanted line's pixels proportionally so its perceived brightness matches a horizontal/vertical line of the same true intensity.

Q2(d) — Koch Curve, 3rd Generation (Triangle $\rightarrow$ Star) [1.5 marks]

Gen 1 (given) = equilateral triangle, 3 segments. Gen 2 (given) = classic Koch rule applied once per edge $\Rightarrow 3 \times 4 = 12$ segments (the 6-pointed star / hexagram).



3rd generation = apply the Koch rule again to *each* of the star's 12 segments $\Rightarrow 3 \times 16 = 48$ segments:



(This is the classic **Koch snowflake**, iteration 2.)

Examination 2021 — Q1(f), Q2(a–b) (Section A)

Question breakdown:

- Q1(f) A Quadratic Koch curve: generation 0 = straight line, generation 1 = square bump (shown). Draw generation 2. (1)
- Q2(a) Line from (1, 1) to (10, 7): (i) raster locations via basic line algorithm; (ii) why is scan conversion slower than expected; (iii) how Bresenham overcomes this; (iv) raster locations via Bresenham. (1.5+1+1+2.5)
- Q2(b) Three major adverse effects of scan conversion? Apply the given lowpass filter kernel to the given 5×5 pixel grid. (2.75)

Q1(f) — Quadratic Koch Curve, Generation 2 [1 mark]

Identical construction to 2023 Q2(d) above (same square-bump rule, one generation number lower on this year's labeling): apply the 5-segment square-bump rule to each

of generation 1's 5 segments $\Rightarrow$ **25 segments** (same 26-point coordinate list shown in §3).

Q2(a) — Line $(1, 1) \rightarrow (10, 7)$: Basic Algorithm vs. Bresenham [6 marks]

$\Delta x = 9, \Delta y = 6 \Rightarrow m = 2/3 \approx 0.667$ ($|m| \leq 1$).

(i) **Basic (direct equation) algorithm** — $b = y_1 - mx_1 = 1 - \frac{2}{3} = \frac{1}{3}$; compute $y = \frac{2}{3}x + \frac{1}{3}$ and round for each integer x :

$(1, 1), (2, 2), (3, 2), (4, 3), (5, 4), (6, 4), (7, 5), (8, 6), (9, 6), (10, 7)$

(ii) **Why slower than expected:** the direct equation requires a floating-point **multiply** (mx) and a **round/floor** operation at every single step — expensive compared to simple integer addition.

(iii) **How Bresenham fixes it:** replaces the per-step multiply with an integer decision variable updated purely by addition/subtraction (see §1.1) — no floating-point arithmetic at all.

(iv) **Bresenham trace:** $\text{Inc}_1 = 2\Delta y = 12$, $\text{Inc}_2 = 2(\Delta y - \Delta x) = 2(6 - 9) = -6$, $d_1 = \text{Inc}_1 - \Delta x = 12 - 9 = 3$.

d	x	y	rule
3	1	1	start
3	2	2	$d > 0 \Rightarrow d+=-6 \Rightarrow -3, y++$
-3	3	2	$d \leq 0 \Rightarrow d+=12 \Rightarrow 9, \text{ same } y$
9	4	3	$d > 0 \Rightarrow d+=-6 \Rightarrow 3, y++$
3	5	4	$y++ \Rightarrow d=-3$
-3	6	4	same $y \Rightarrow d=9$
9	7	5	$y++ \Rightarrow d=3$
3	8	6	$y++ \Rightarrow d=-3$
-3	9	6	same $y \Rightarrow d=9$
9	10	7	$y++$

$(1, 1), (2, 2), (3, 2), (4, 3), (5, 4), (6, 4), (7, 5), (8, 6), (9, 6), (10, 7)$

Identical to the correctly-rounded basic-algorithm result in (i) — as expected, since Bresenham is just an efficient reformulation of "pick the nearest pixel to the true line," not a different rule.

Q2(b) — Three Adverse Effects + Lowpass Filter [2.75 marks]

Three adverse effects: staircase/jaggies, unequal brightness of slanted lines, the picket fence problem (see §1.4).

Lowpass filtering: new value = $\sum(\text{kernel weight}) \times (\text{corresponding original pixel value})$, kernel centered on each pixel. For the given 3×3 kernel (top neighbor weight 3, bottom neighbor weight 2, all else 0) applied to the interior of the given 5×5 grid:

$$\text{new}(r, c) = 3 \times \text{grid}(r-1, c) + 2 \times \text{grid}(r+1, c)$$

$$\begin{array}{lll} 3(0) + 2(25) = \mathbf{50} & 3(0) + 2(92) = \mathbf{184} & 3(0) + 2(31) = \mathbf{62} \\ 3(13) + 2(44) = \mathbf{127} & 3(31) + 2(74) = \mathbf{241} & 3(46) + 2(94) = \mathbf{326} \\ 3(25) + 2(0) = \mathbf{75} & 3(92) + 2(0) = \mathbf{276} & 3(31) + 2(0) = \mathbf{93} \end{array}$$

Method is what's graded, not these exact numbers — re-read the kernel and grid values off your own copy of the paper before the exam; the mechanical rule (multiply-and-sum a centered kernel over the neighborhood) is what must be memorised.

Examination 2020 — Q2(d), Q3(b–c), Q4(c) (Group-A)

Question breakdown:

- Q2(d) Could you fill the given arrow shape using the Flood-Fill algorithm with 8-connected pixels? Justify. (2.5)
 Q3(b) Show raster locations while drawing a line using DDA from (1, 1) to (5, 8). (3.75)
 Q3(c) Find raster locations using Bresenham's algorithm, line from (0, 0) to (8, 5). (4)
 Q4(c) Show 8-way symmetry of a circle. (1)

Q2(d) — Flood-Fill an Arrow, 8-Connected [2.5 marks]

Same reasoning as 2023 Q2(c) (§3): an arrow shape has slanted edges (the head's diagonal sides), so its scan-converted boundary is a single-pixel staircase that is only 4-connected along its length. Running **flood-fill with 8-connected** logic lets the fill step diagonally through the corner-gaps in that staircase, **leaking out of the arrow at its diagonal edges** — so no, it cannot be reliably filled this way without first thickening the boundary or switching the fill to 4-connected.

Q3(b) — DDA, (1, 1) $\rightarrow$ (5, 8) [3.75 marks]

$\Delta x = 4, \Delta y = 7 \Rightarrow m = 7/4 = 1.75$ ($|m| > 1$, so step y by 1, $x_{i+1} = x_i + 1/m = x_i + 4/7$).

y	x (exact)	pixel x	pixel
1	1.000	1	(1, 1)
2	1.571	2	(2, 2)
3	2.143	2	(2, 3)
4	2.714	3	(3, 4)
5	3.286	3	(3, 5)
6	3.857	4	(4, 6)
7	4.429	4	(4, 7)
8	5.000	5	(5, 8)

(1, 1), (2, 2), (2, 3), (3, 4), (3, 5), (4, 6), (4, 7), (5, 8)

Q3(c) — Bresenham, $(0, 0) \rightarrow (8, 5)$ [4 marks]

$\Delta x = 8, \Delta y = 5 \Rightarrow m = 0.625$ ($0 < m < 1$). $\text{Inc}_1 = 2\Delta y = 10$, $\text{Inc}_2 = 2(\Delta y - \Delta x) = 2(5 - 8) = -6$, $d_1 = \text{Inc}_1 - \Delta x = 10 - 8 = 2$.

d	x	y	rule
2	0	0	start
2	1	1	$d > 0 \Rightarrow d+ = -6 \Rightarrow -4, y++$
-4	2	1	$d \leq 0 \Rightarrow d+ = 10 \Rightarrow 6$, same y
6	3	2	$d > 0 \Rightarrow d+ = -6 \Rightarrow 0, y++$
0	4	2	$d \leq 0 \Rightarrow d+ = 10 \Rightarrow 10$, same y
10	5	3	$d > 0 \Rightarrow d+ = -6 \Rightarrow 4, y++$
4	6	4	$d > 0 \Rightarrow d+ = -6 \Rightarrow -2, y++$
-2	7	4	$d \leq 0 \Rightarrow d+ = 10 \Rightarrow 8$, same y
8	8	5	$y++$

(0, 0), (1, 1), (2, 1), (3, 2), (4, 2), (5, 3), (6, 4), (7, 4), (8, 5)

Q4(c) — 8-Way Symmetry of a Circle [1 mark]

See §1.2 — one octant point (x, y) mirrors to 7 others: $(x, y), (y, x), (-y, x), (-x, y), (-x, -y), (-y, -x), (x, -y)$.

Quick Summary — Exam Patterns for Chapter 3

Pattern	Years	Method to memorise
DDA vs. Bresenham line trace	2020, 2021, 2024	Table walk: DDA rounds $y+=m$ each step; Bresenham updates integer d by Inc_1 or Inc_2
Bresenham/DDA conceptual (why faster)	2021, 2023	No floating multiply/round in Bresenham; integer add/shift only
Midpoint circle raster trace (octant + symmetry)	2022, 2023	$p_1 = 1-r$; $p < 0 \Rightarrow p+=2x+3$; else $p+=2(x-y)+5$, $y--$
8-way symmetry (theory only)	2020, 2022	$(x, y), (y, x), (-y, x), (-x, y), (-x, -y), (-y, -x), (y, -x), (x, -y)$
Slanted-line dimness (why / how to fix)	2021, 2022, 2023	$\sqrt{2}$ vs. 1 unit pixel spacing; fix via area/super-sampling anti-aliasing
Flood-fill 4/8-connected leak reasoning	2020, 2023	Same connectivity for boundary & fill is self-contradictory on diagonal edges
Koch curve, draw next generation	2021, 2022, 2023	Square-bump: $\times 5$ segs/edge/iter; classic triangular: $\times 4$ segs/edge/iter

Exam-day strategy for the Scan Conversion cluster: this question is entirely mechanical once the two decision-variable recurrences (line, circle) are memorised cold — there is no derivation required in the exam, only correct step-by-step table execution. Budget the marks accordingly: a full trace (line or circle) is usually worth 3–4 marks for maybe 8–10 minutes of careful arithmetic; the conceptual sub-parts (why dimmer, why Bresenham is faster, flood-fill leak reasoning) are 1–2 mark free points if the one-line reasons above are memorised.